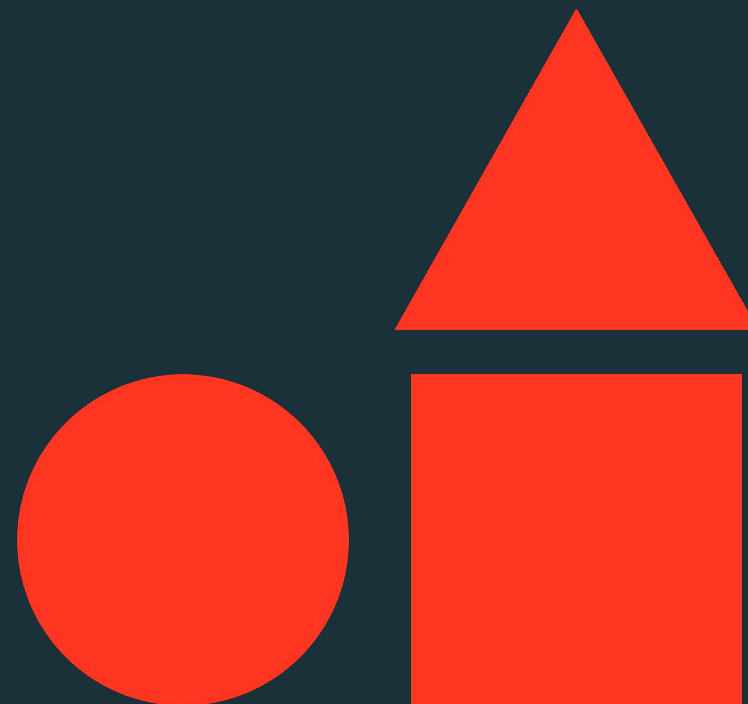


Accelerating Innovation + Self-Service

Driving Platform Governance and
Efficiency at Scale

RPM
2026-06



Autonomy requires boundaries. Velocity requires standards.

By strictly **isolating** the Platform State from the Delivery State

And establishing clear **handshake protocols** for Network, Security, and Global Governance

We enable Enterprise IT to protect the foundation, while Business Units **iterate** on data products at the **speed** of their markets



Common Controls

Identity, workspace folders, Unity Catalog guardrails, governed tags, and service-principal promotion are mandatory across the board.

Flexible Delivery

Autonomous teams self-deploy; emerging teams prototype in a safe sandbox and rely on assisted promotion.

Value Realization

Success is measured by prototype-to-promotion conversion, asset reuse, and cleanup discipline, not just lines of code deployed.

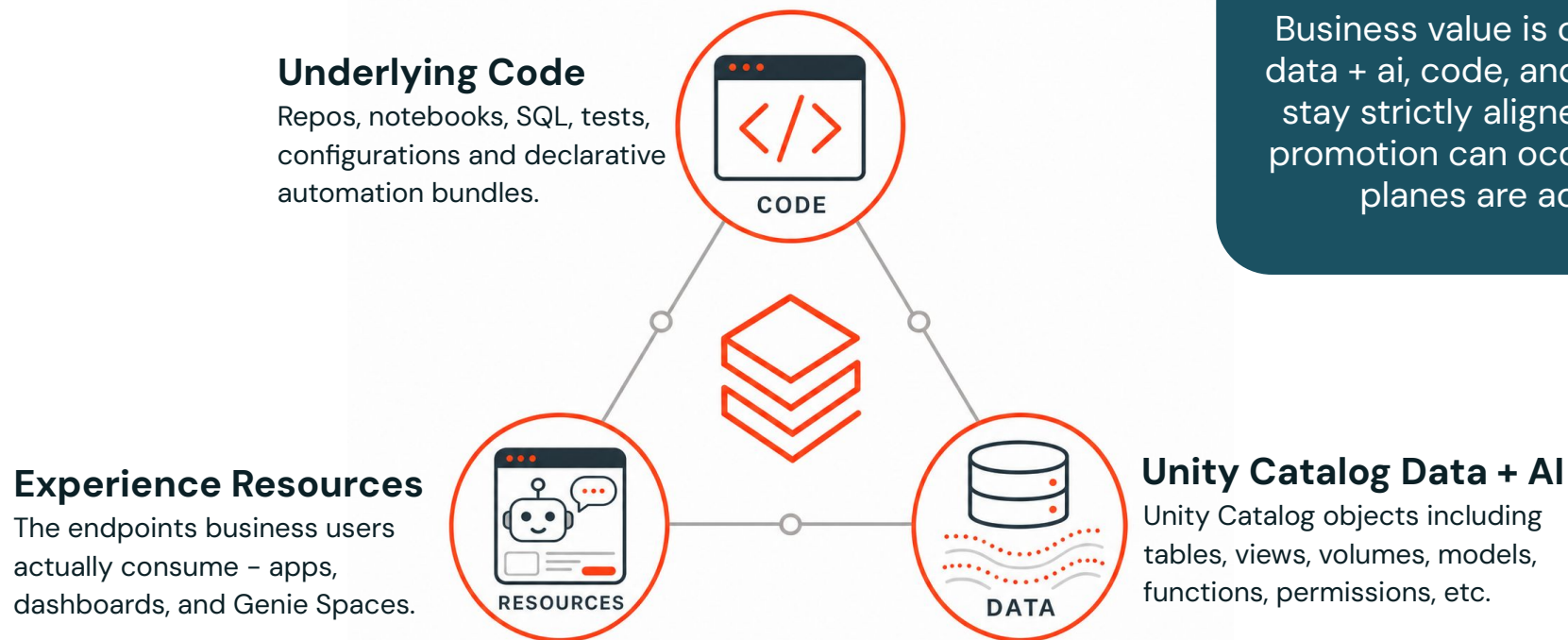
Unified Governance, Dual Delivery Patterns

RPM can standardize a single governance control plane while routing teams into two distinct delivery patterns based on their maturity.



Pillars Across Patterns

Requirements Regardless of Maturity Level



Business value is created only when data + ai, code, and user experiences stay strictly aligned. No production promotion can occur unless all three planes are accounted for.

Risk

If fragmented, teams promote code but forget the business-facing experience or discovery metadata.

Reward

If aligned, every promoted asset is findable, governed, and tied directly to a business owner.



Dual Operating Model

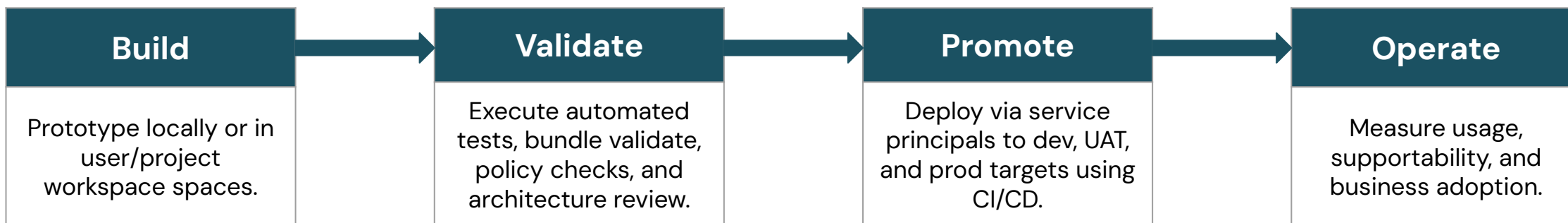
Aligning Patterns to the Team Maturity

	Autonomous	Assisted
Team Profile	Established data product domains (e.g., Market Insights Supply Chain) with strong engineering discipline.	Less mature or newly enabled domains (e.g., broad Business Users, Finance Analysts) focused on rapid enablement.
CI/CD & Delivery Ownership	Team owns repo strategy, automated tests, release cadence, and runtime support using their own CI/CD platform.	Team prototypes freely in safe boundaries; enterprise central teams review and promote approved assets via bundles.
Architectural Boundary	Provisioned their own catalogs across deployment environments with full ownership of the catalog.	Uses a centrally-managed catalogs where user-based schemas are provisioned with full ownership of just the schema in the dev environment.
Accelerating Value Realization	Fastest path to scale and throughput with clear accountability.	Low-friction experimentation without uncontrolled production risk.



Single Deployment Contract

Consistency Across Production Assets + Team Maturity



The Four Required "Knowns" for Deployment

Every approved change must strictly map to:

Workspace

Catalog
Boundary

Compute
Configuration

Tagging and
Ownership



Principles of the Shared Platform

Partnering for Scaled Success

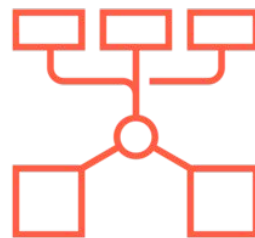
Standardize Once

Enterprise Architects and Platform Engineers codify identity, network, and the workspace shell. Decisions are visible, implementation is traceable.



Decouple Delivery

Business Units own their data products. Terraform manages the shared plumbing; Declarative Automation Bundles manage the workloads. Separate state files guarantee safety.



Joint Design Patterns

Routine domain changes happen at BU speed. Central IT only steps in for global policies, front-door security, or core corporate network paths.



The Shared Model

Separation for Stability and Scale



Enterprise IT (*The Plumbing*)

Core Mandate:

Centralize the Foundation to contain the blast radius and risk.

Automation Focus:

Terraform for account, identity, shared AWS networking, and workspace shells.

Cadence:

Stable, coordinated, and governed.



Business Unit (*The Products*)

Core Mandate:

Curate data and analytics products without IT bottlenecks.

Automation Focus:

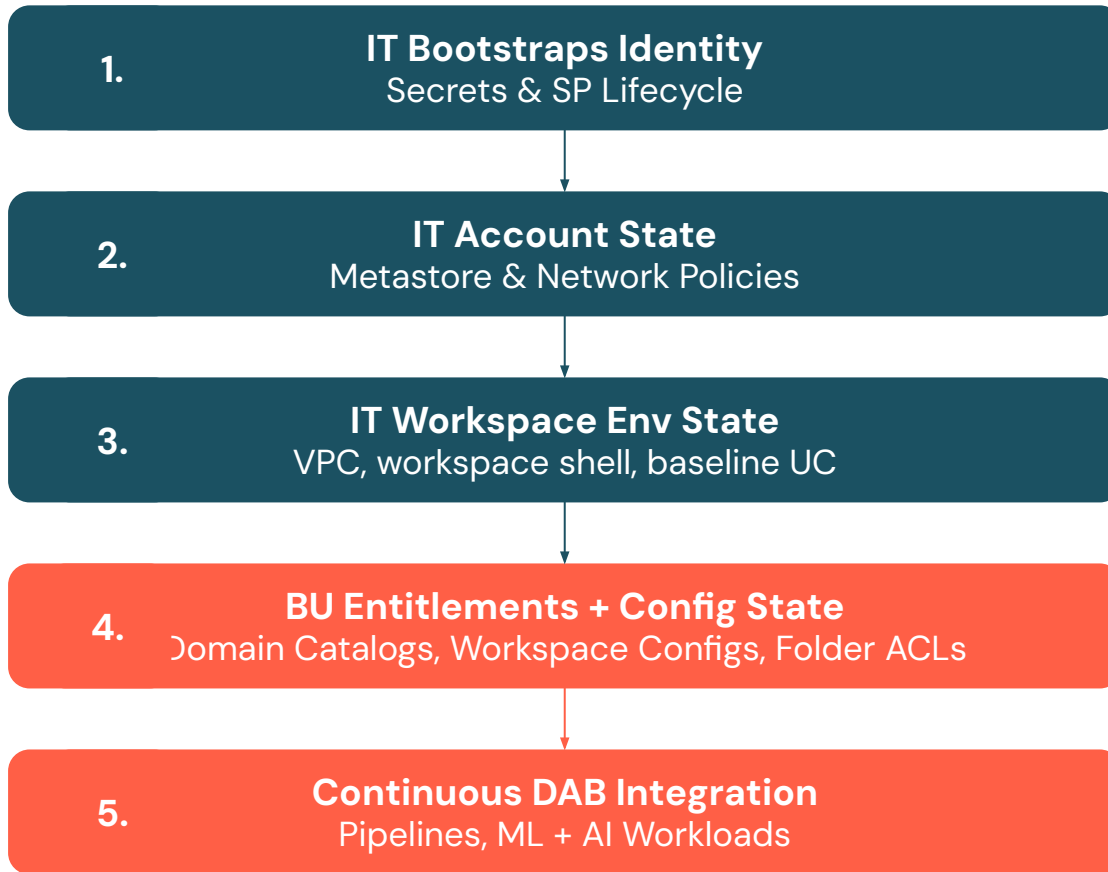
Declarative Automation Bundles (DABs) for pipelines, MLflow, and jobs. Additive BU Terraform for entitlements and configs.

Cadence:

High-velocity, iterative, and decoupled from platform states.



Pipeline Sequence + Operations Order



Steps 1–3 are owned by Ent IT

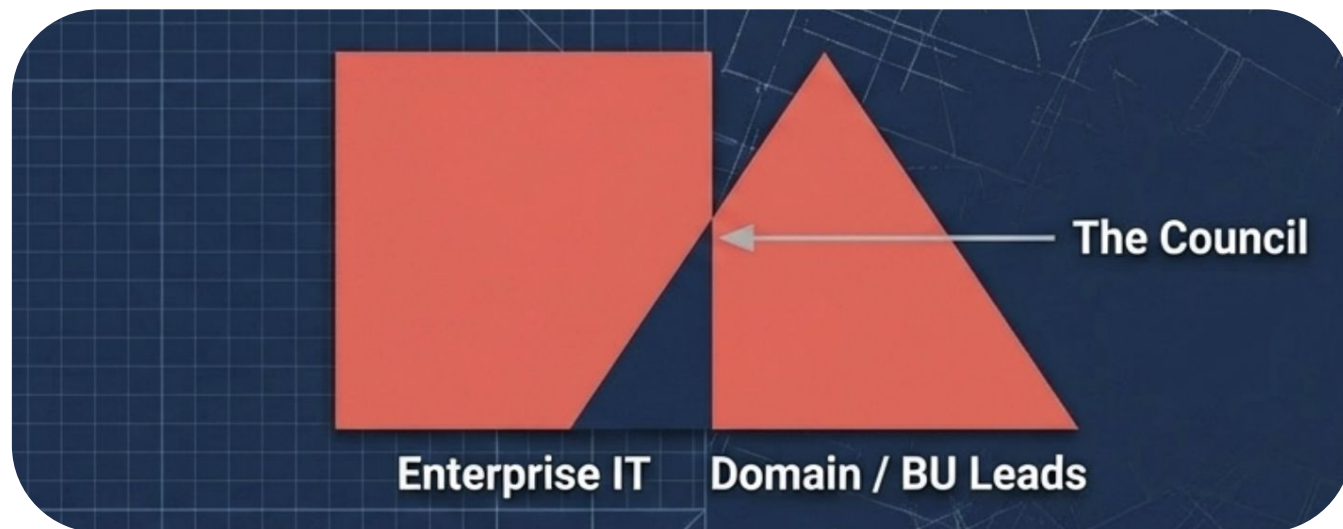
Steps 4–5 are owned by Business Units

Parallel BU Pipelines can run after the same workspace apply without ever blocking each other. BUs never recreate the VPC or other low-level infrastructure components.



Shared Operating Model

Data Governance Council for Collaboration



The council acts as the centralized authority, forging enterprise-wide standards while delegating operational agility to the business domains.

Establish cross-BU global tags
(ie. PII, Geography)

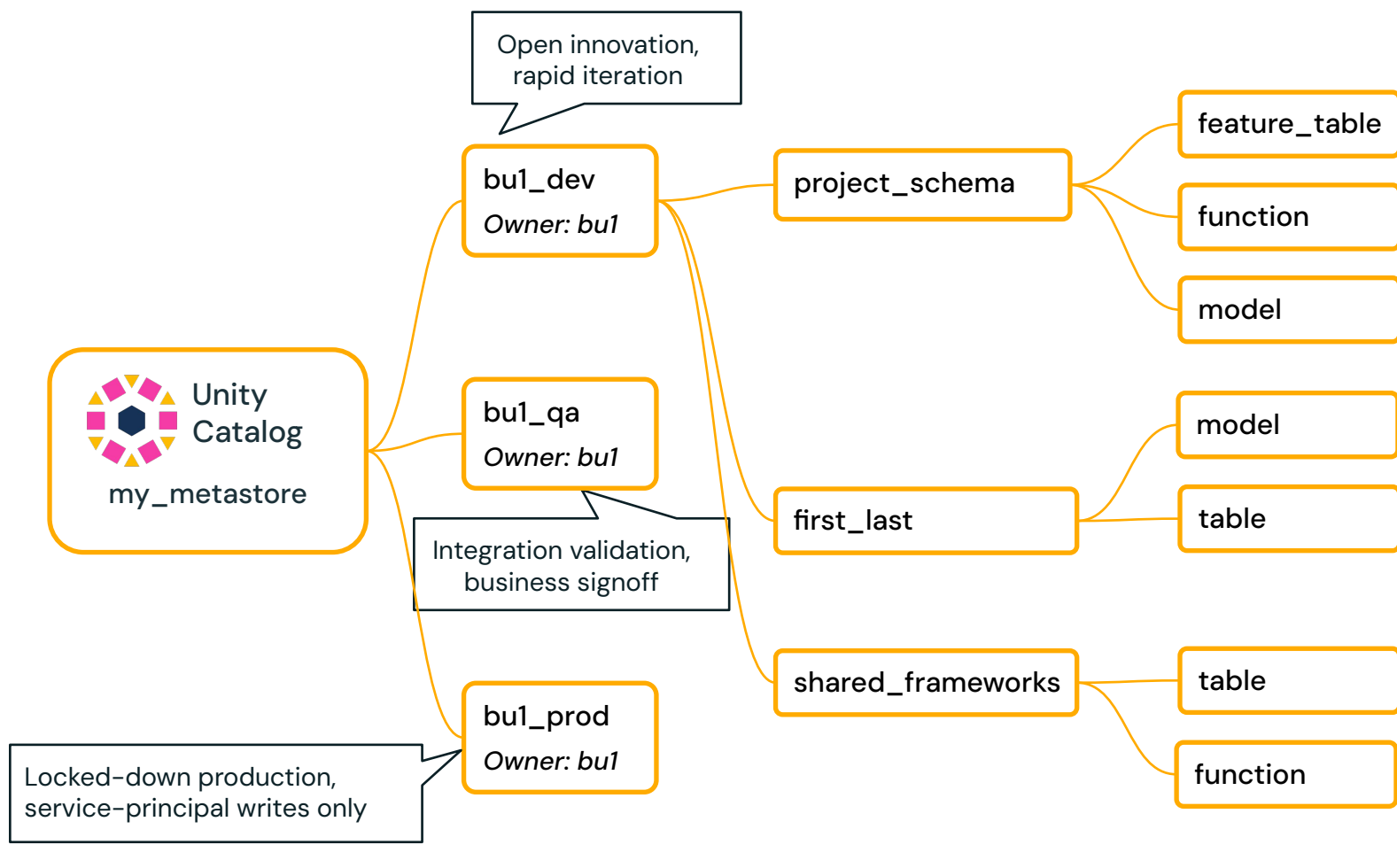
Align domain-specific governed
keys to prevent overlap.

Oversee the materialized audit
snapshot for remediation.



Autonomous: Catalog Ownership

Mature Teams that Need Strong Ownership and Clean SDLC Boundaries



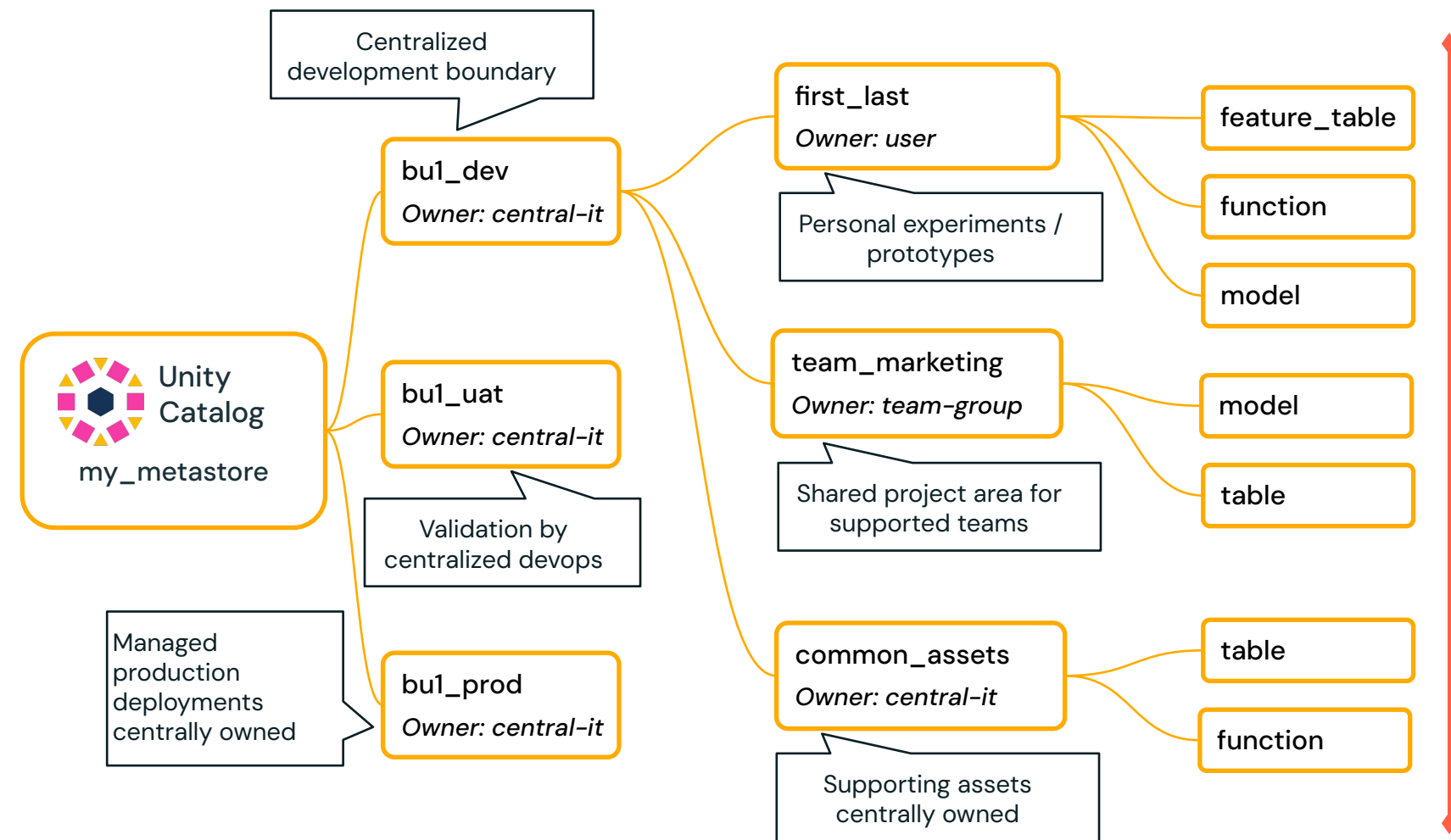
Key Characteristics

- Privilege hierarchy is crystal clear.
- Production writes are reserved strictly for automation identities.
- Watchout: Can create catalog sprawl if assigned to immature teams too early.



Assisted: BU Catalog

Growing Teams that Need Safe Self-Service and Central Oversight



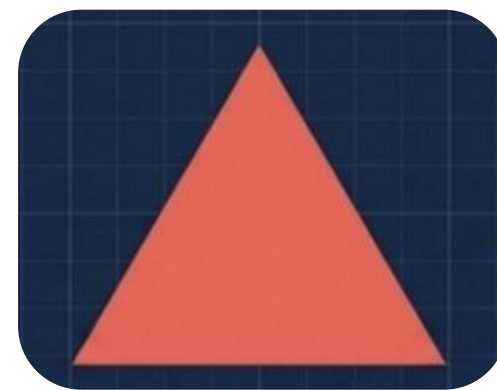
Key Characteristics

- Every asset is isolated by schema owner without needing a full catalog per user.
- Enterprise platform teams can monitor create/update/delete behavior centrally.
- Requires strict naming, retention, and promotion rules.



Segregation of Duties

Permission Cascade



Enterprise IT

Ultimate Authority

Holds Metastore Admin. Defines the highest-level parameters and system infrastructure.

Governance Leads

Delegated Authority

Holds `CREATE` for governed tags and `MANAGE` for definitions and allowed values.

Business Users

Operational Agility

Holds `ASSIGN` on specific tags and `APPLY TAG` on directly managed assets.



Govern Once, Signal Everywhere

Standardized Tags for Control at Scale

Controlled Metadata: Governed Tags

Account-level governed tags with approved values and controlled assignment rules.

Governed Tags Answer: What is this asset, what state is it in, and how should it be filtered, reviewed, or automated?

Business Experience: Discover Domains

A curated business grouping with technical owner, business owner, and a page that helps people find trusted assets.

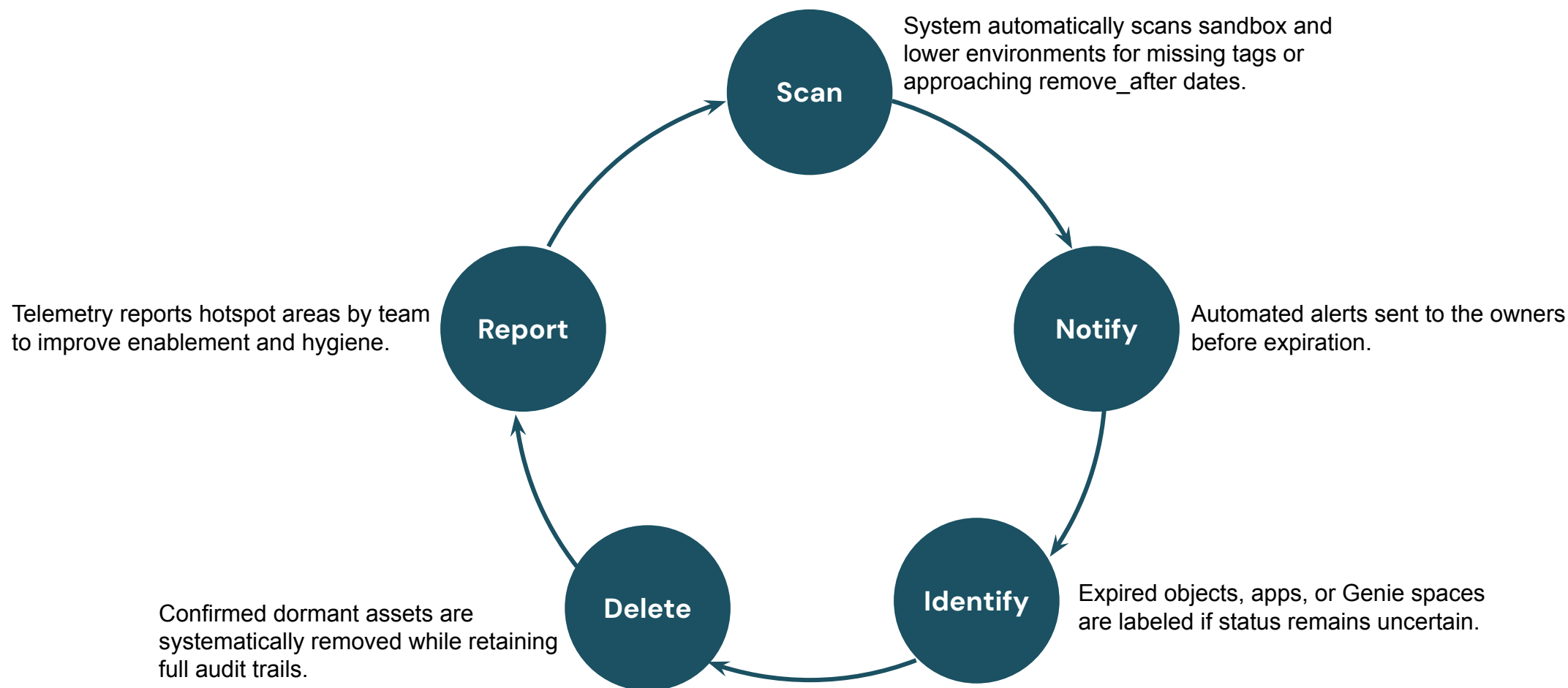
Domains Answer: Which business area does this belong to and who is accountable for it?

Domains organize assets for business **discovery** and **stewardship**, while governed tags provide the **controlled** metadata model that makes those domains work **consistently** across Unity Catalog and workspace assets.



Active Governance

Govern, Review, Retire: Clean up inactive assets



Observability at Scale

Driving Governance from Evidence

Capture

Ingest from system tables the audit logs, Lakeflow runs, and query history.

Monitor workspace activity, access events, and permission changes.

Correlate

Join platform events to the governed tags, owners, domains, and deployment history.

Link system health and pipeline status to business adoption metrics (e.g., Genie usage frequency).

Act

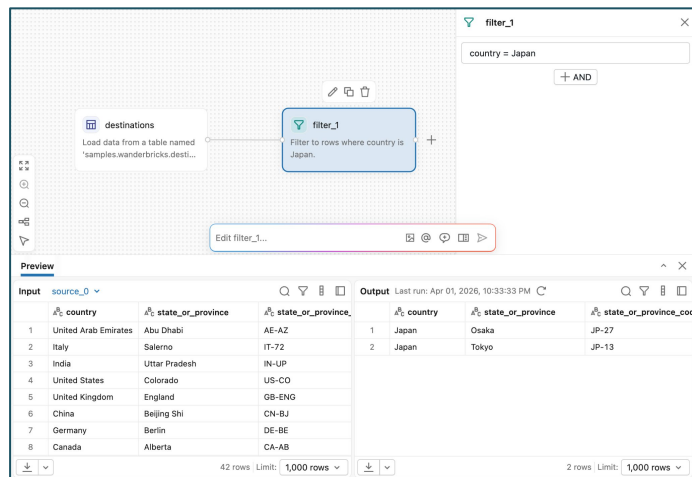
Drive governance workflows automatically.

Route incidents, revoke access, trigger the quarantine cleanup loop, and prioritize recertification of high-value assets.



Accelerating Innovation

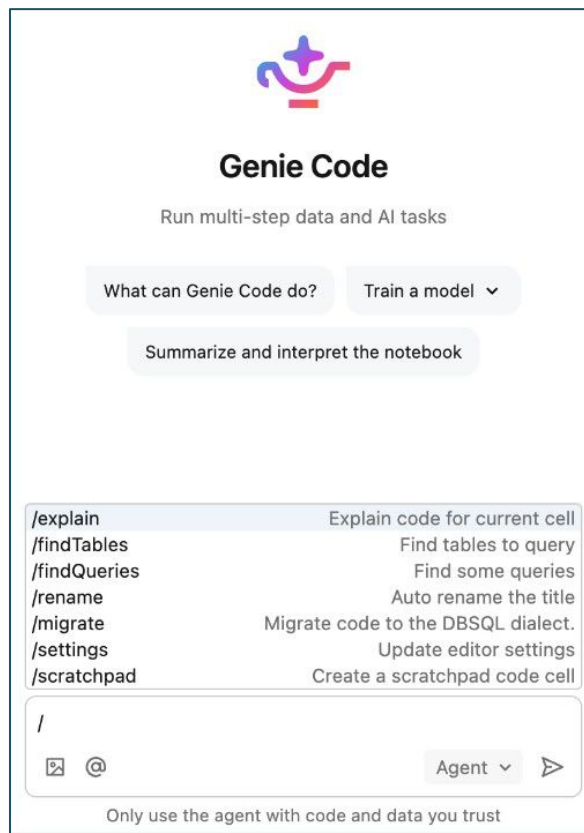
Enabling Users on Modern Tools to Streamline the Transition



Lakeflow Designer

Replaces visual ETL build patterns.

Outputs native pipelines that are governed and observable. Eliminates the "rewrite gap" between business prototype and engineering production.



Genie Code

Reduces technical friction for users moving off external tools.

Generates SQL/Python from plain language. Not a replacement for visual ETL, but a bridge for SQL users to become productive natively.



Curated Consumption Experiences

Treat Apps, Dashboards, and Genie Spaces as managed surfaces.
Placement drives governability.

Autonomous Pattern: Central IT Provides Guardrails

- Provides the deployment contract: folders, targets, permissions, and tags.
- Enforces identity, least privilege, auditability, and policy compliance.
- Monitors drift, exceptions, and risk rather than owning each deployment.
- Supports teams with patterns, reviews, and escalation paths.
- Lets mature teams own promotion and support inside the shared control framework.

Assisted Pattern: Central IT is an Operator

- Provides managed folders, warehouses, service principals, and access patterns.
- Reviews candidates, validates dependencies, and approves promotion.
- Owns production deployment, support, and access changes.
- Applies lifecycle and review controls to separate experimental from operational assets.
- Partners with the domain but keeps day-to-day production control.



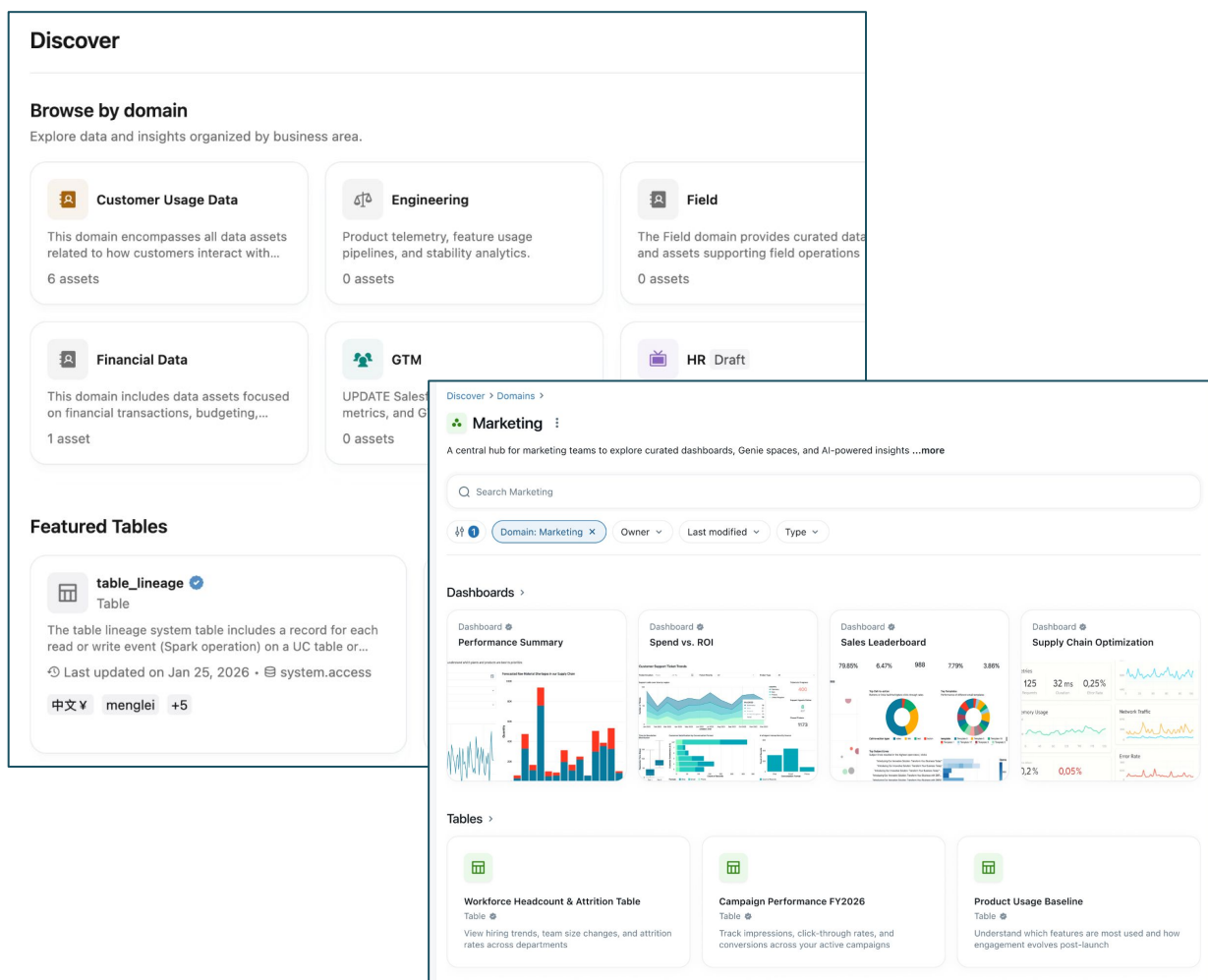


Appendix



Business Discovery

Turning Inventory Into a Map for Discovery and Self-Service



Domains

Account-level groupings (e.g., Finance, Supply Chain) built on governed tags that organize published assets.

Certified Trust Signals

Trust tags surface high-value, production-ready assets to the top of consumer search results.

The Discover Experience

A curated browsing interface across tables, dashboards, and Genie spaces that hides technical complexity from business users.



Tagging Architecture

Controlled Tags for Governance, Lifecycle Management and Observability

Tag Category (Purpose)	Examples & Function
Domain (Business Context)	ie. marketing, supply_chain Groups assets on Domain pages
Certification (Trust Signal)	Apply system tags to objects to mark them as <code>certified</code> or <code>deprecated</code>
Recertification (Action Triggers)	ie. <code>review_after</code> , <code>remove_after</code> dates
And more..	ie. <code>sensitivity</code> , <code>refresh_sla</code> , etc.

The Rule: Sandbox prototypes remain discoverable mainly to builders and curators. Promoted, certified assets are clearly identifiable across Domains.



Automation Surfaces and Personas

Mapping the Core Personas to their Primary Codebases

Platform Discipline (Enterprise IT)

Roles:

Enterprise / Platform Architect &
Technical Platform Engineer

Surface:

Platform Terraform Codebase

Responsibilities:

Account hooks, metastore
baseline, security perimeters,
CI/CD templates.

The Bridge (BU Liaison)

Roles:

BU Workspace Admin & DevOps
Engineers

Surface:

BU-Owned Terraform & Platform
Outputs

Responsibilities:

Translates steward requirements
into code; orchestrates
entitlements, domain catalogs
and workspace configs.

Product Delivery (Data / ML Teams)

Roles:

BU Data Stewards & Data/ML
Engineers

Surface:

Declarative Automation Bundles
(databricks.yml)

Responsibilities:

Delivery of jobs, ML workflows,
dashboards via project CI/CD. No
direct interaction with Platform
Terraform.



Delivery Scenarios

Defining When to Run Autonomously and When to Collaborate

Track 1: BU Stand-Alone Execution (Routine)

Scenarios Examples:

New user access, domain catalogs, external locations, Unity Catalog permissions, compute configurations, workspace ACL configurations, data + ai jobs.

Process:

Executed entirely via BU Terraform and Bundle pipelines within existing guardrails. No Enterprise IT collaboration required.

Track 2: Joint Execution (The Handshake)

Scenarios Examples:

On-prem/Partner Routing, Front-Door Rules, Global Cross-BU Data Sharing.

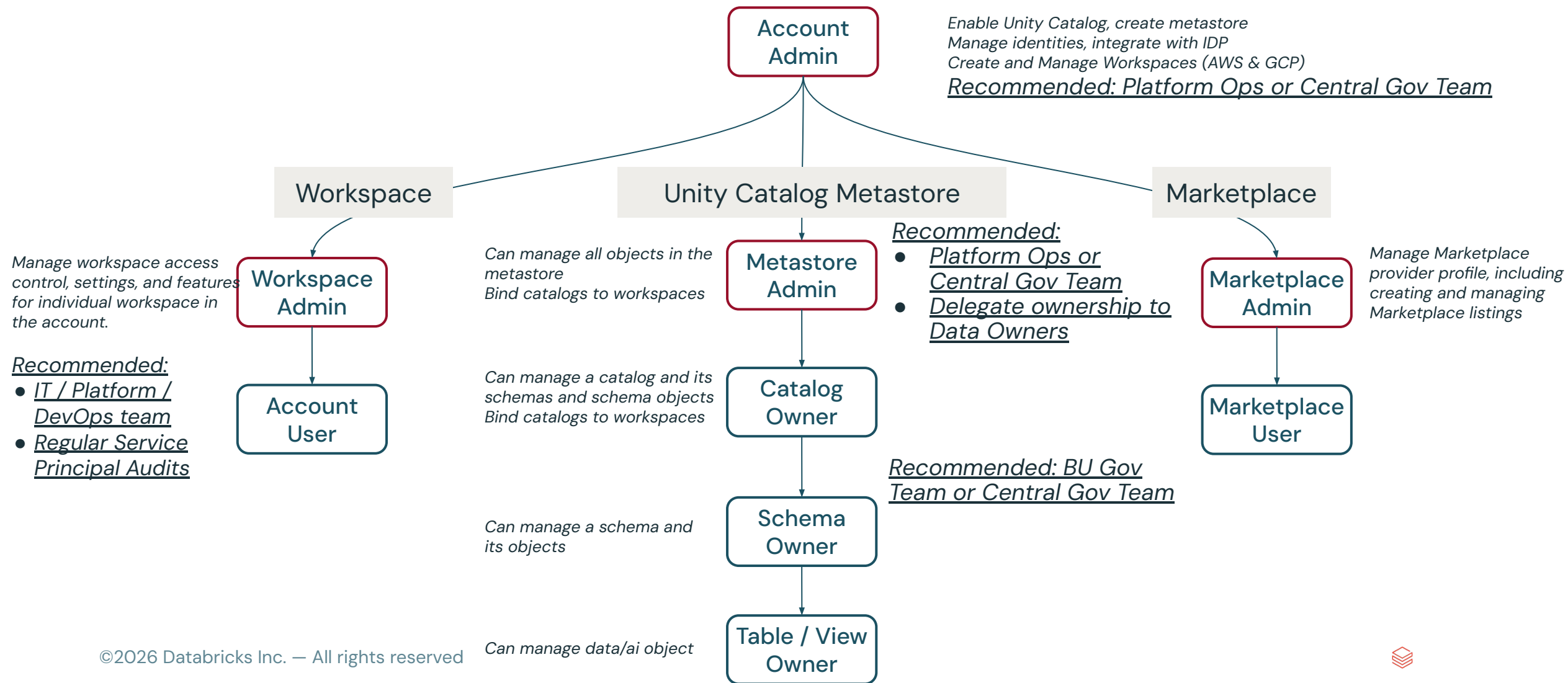
Process:

Shared network, account-wide policy, or front-end security is triggered. Demands a synchronized handoff between Platform Engineer and BU Liaison.



Administrative Alignment

Identifying Admins for Distributed Governance



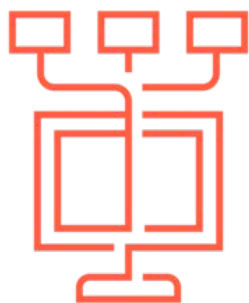
A Deeper Look: Dimensions of Data

Grouping Data to Facilitate Discovery and Adoption

Environment

The deployment stage of the data across the development lifecycle

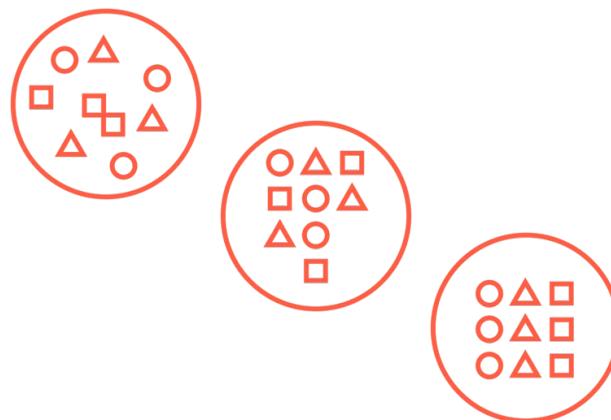
- Dev
- QA
- UAT
- Production
- ...



Validation

The quality of the data – is the data prepared for consumption?

- Bronze
- Silver
- Gold
- ...



Business Unit

The team responsible for the data

- Central IT
- Finance
- HR
- Marketing
- Research & Development
- ...



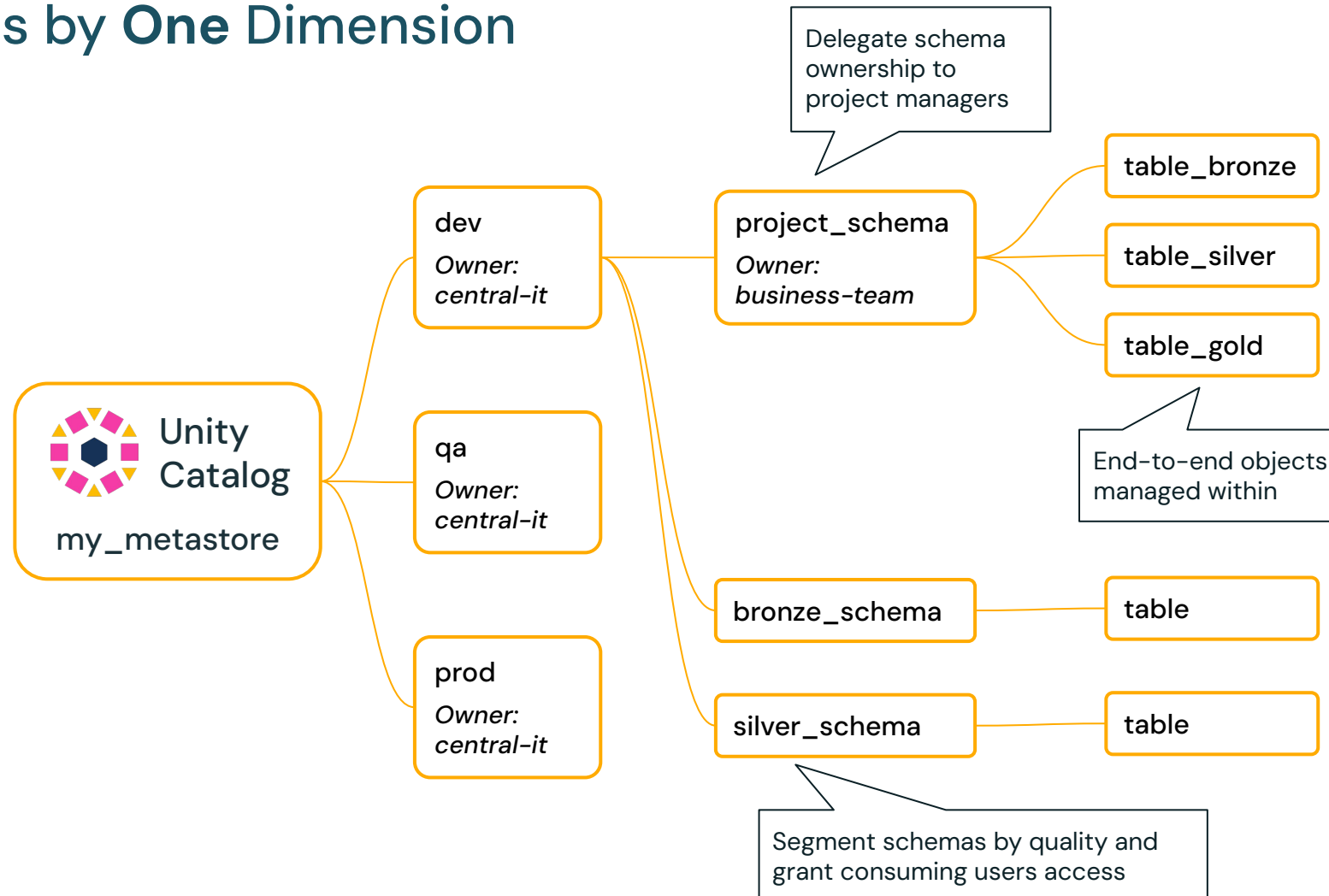
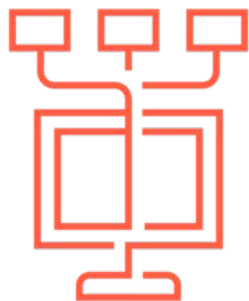
Practical Example: Organizing Catalogs

Grouping Data + AI Assets by One Dimension

Environment

The deployment stage of the data across the development lifecycle

- Dev
- QA
- UAT
- Production
- ...



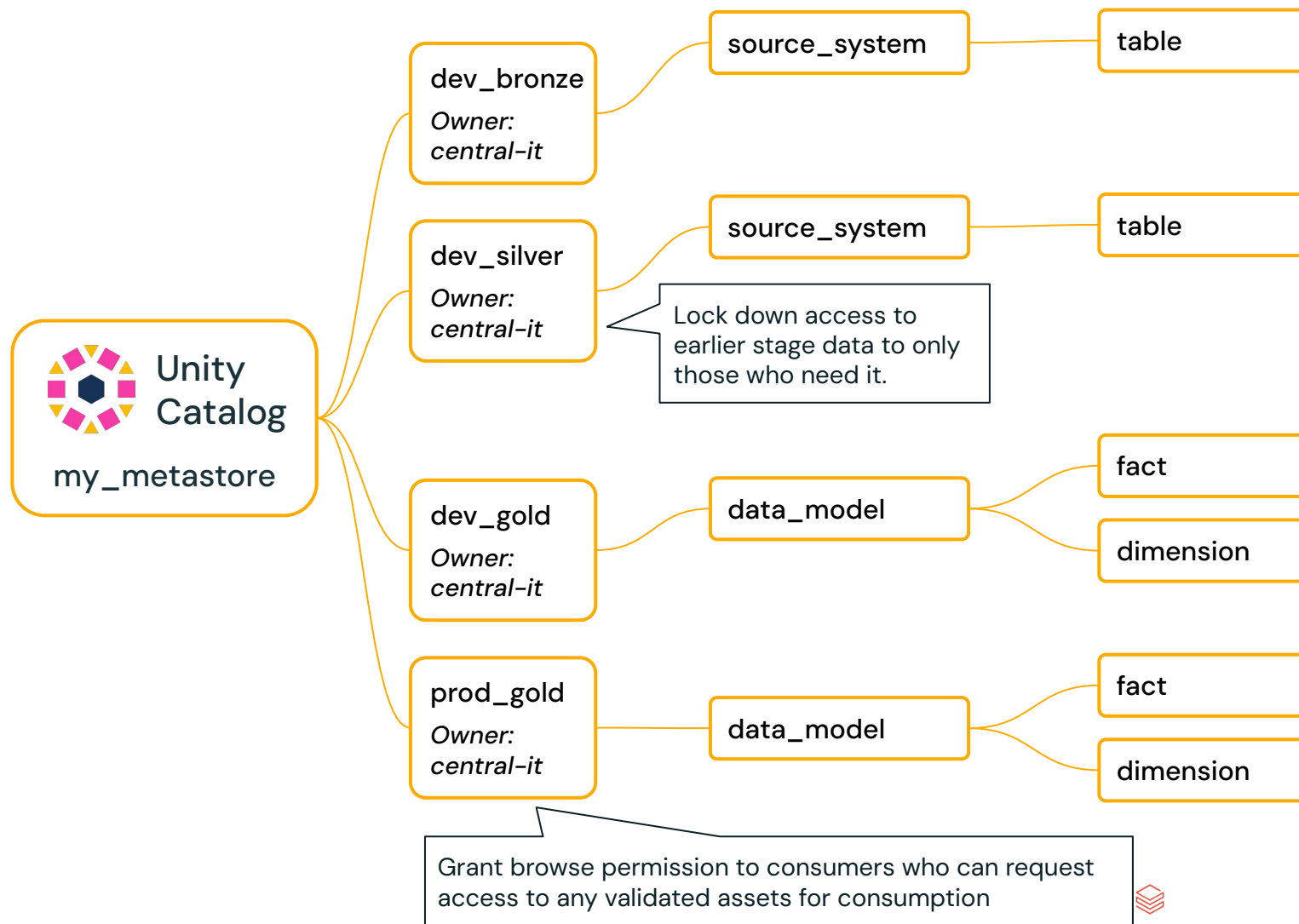
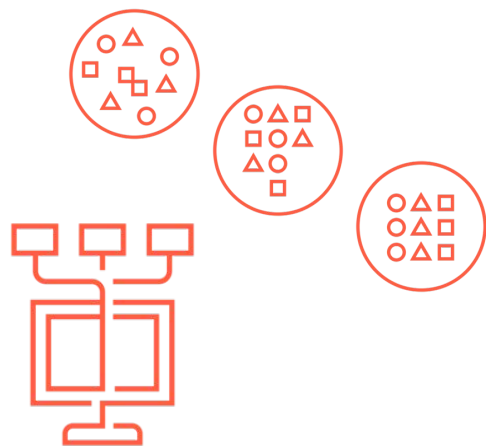
Practical Example: Organizing Catalogs

Grouping Data + AI Assets by Two Dimensions

Environment + Validation

The deployment stage of the data across the development lifecycle.

The quality of the data – is the data prepared for consumption?



Practical Example: Organizing Catalogs

Hybrid Approach for Grouping Data + AI Assets

Environment, Validation and Business Units

The deployment stage of the data across the development lifecycle.

The quality of the data – is the data prepared for consumption?

The team responsible for the data

